

AFRILANG Stdlib

std/c/statquartile

Français. Bibliothèque AFRILANG 'std/c/statquartile' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : q1, q2, q3, iqr, quartiles, fiveNum, outlierFence.

```
'import "std/c/statquartile"' · 'use statquartile'
```

```
- 'q1(v list number) → number' - 'q2(v list number) → number' - 'q3(v  
list number) → number' - 'iqr(v list number) → number' - 'quartiles(v  
list number) → list number' - 'fiveNum(v list number) → list number'  
- 'outlierFence(v list number) → list number'
```

English. AFRILANG library 'std/c/statquartile' (tier complex, category complex). Exports 7 function(s): q1, q2, q3, iqr, quartiles, fiveNum, outlierFence.

```
'import "std/c/statquartile"' · 'use statquartile'
```

```
- 'q1(v list number) → number' - 'q2(v list number) → number' - 'q3(v  
list number) → number' - 'iqr(v list number) → number' - 'quartiles(v  
list number) → list number' - 'fiveNum(v list number) → list number' -  
'outlierFence(v list number) → list number'
```

Catégorie / Category	Modules complexe (c/)	
Niveau / Tier	complexe	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/statquartile' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : q1, q2, q3, iqr, quartiles, fiveNum, outlierFence.

'import "std/c/statquartile"' · 'use statquartile'

```
- `q1(v list number) → number`
- `q2(v list number) → number`
- `q3(v list number) → number`
- `iqr(v list number) → number`
- `quartiles(v list number) → list number`
- `fiveNum(v list number) → list number`
- `outlierFence(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statquartile"
use statquartile
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- q1(v list number) → number•
q2(v list number) → number•
q3(v list number) → number•
iqr(v list number) → number•
quartiles(v list number) → list•
fiveNum(v list number) → list•
outlierFence(v list number) → list

4. Exemples d'utilisation

```
import "std/c/statquartile"
use statquartile
```

```
create result = q1(/* args */)
say result
```

```
create result = q2(/* args */)
say result
```

```
create result = q3(/* args */)
say result
```

```
create result = iqr(/* args */)
say result
```

```
create result = quartiles(/* args */)
say result
```

```
create result = fiveNum(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/statquartile"`` en tête de fichier.
- Lancez ``afri-lang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module statquartile

```
function q1(v list number) returns number
end
```

```
function q2(v list number) returns number
end
```

```
function q3(v list number) returns number
end
```

```
function iqr(v list number) returns number
end
```

```
function quartiles(v list number) returns list number
end
```

```
function fiveNum(v list number) returns list number
end
```

```
function outlierFence(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/statquartile' (tier complex, category complex). Exports 7 function(s): q1, q2, q3, iqr, quartiles, fiveNum, outlierFence.

```
'import "std/c/statquartile"' · 'use statquartile'
```

```
- `q1(v list number) → number`  
- `q2(v list number) → number`  
- `q3(v list number) → number`  
- `iqr(v list number) → number`  
- `quartiles(v list number) → list number`  
- `fiveNum(v list number) → list number`  
- `outlierFence(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statquartile"  
use statquartile
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- q1(v list number) → number•
q2(v list number) → number•
q3(v list number) → number•
iqr(v list number) → number•
quartiles(v list number) → list•
fiveNum(v list number) → list•
outlierFence(v list number) → list

4. Usage examples

```
import "std/c/statquartile"  
use statquartile
```

```
create result = q1(/* args */)   
say result
```

```
create result = q2(/* args */)   
say result
```

```
create result = q3(/* args */)   
say result
```

```
create result = iqr(/* args */)   
say result
```

```
create result = quartiles(/* args */)   
say result
```

```
create result = fiveNum(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/statquartile"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module statquartile

```
function q1(v list number) returns number
end
```

```
function q2(v list number) returns number
end
```

```
function q3(v list number) returns number
end
```

```
function iqr(v list number) returns number
end
```

```
function quartiles(v list number) returns list number
end
```

```
function fiveNum(v list number) returns list number
end
```

```
function outlierFence(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/statquartile"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/statquartile/>

Source (extrait)

```
module statquartile

function q1(v list number) returns number
end

function q2(v list number) returns number
```

```
end

function q3(v list number) returns number
end

function iqr(v list number) returns number
end

function quartiles(v list number) returns list number
end

function fiveNum(v list number) returns list number
end

function outlierFence(v list number) returns list number
end
end
```